

Imitation Learning for Neural Network Autopilot in Fixed-Wing Unmanned Aerial Systems

Daksh Shukla, Shawn Keshmiri and Nicole Beckage

Abstract—This research identifies the feasibility of training an artificial neural network (ANN) autopilot, using supervised learning techniques, including an imitation learning framework known as the data aggregation set (Dagger) algorithm. The ANN autopilot aims to mimic a unified guidance, navigation & control (GNC) system to fly a fixed-wing UAS. Utilizing high-fidelity nonlinear 6-DOF aircraft simulations, it is shown that several modifications in existing imitation learning techniques must be considered. Dagger algorithm when applied sequentially to augment data along the desired flight trajectory to train the ANN autopilot is unable to generalize across turning and straight-line maneuvers, and hence cannot learn to fly stably, eventually terminating the simulation. Monte-Carlo methods when incorporated into the Dagger algorithm allow for sampling of random data along the desired flight trajectory, proved to be effective to train the ANN autopilot to fly indefinitely with acceptable tracking errors, however undesirable low frequency oscillations were observed in control input and aircraft states. The oscillatory behavior is dealt with by introducing a time-based moving window along the trajectory for data addition in conjunction with the standard Dagger algorithm. Different variations of Dagger algorithms are tested and compared using closed-loop 6-DOF flight simulations. 3D trajectory tracking and stability in aircraft states are evaluated with evidence supporting the idea that an ANN autopilot can behave as a unified GNC system and fly a fixed-wing UAS.

Keywords: Machine Learning, Neural Networks, Dagger, UAS, Guidance, Navigation, Control, Monte-Carlo, McDagger, MwDagger.

I. INTRODUCTION

A fully autonomous flight control system for an aerospace vehicle involves three major algorithms: guidance, navigation and control (GNC). The whole system of algorithms is jointly implemented in a feedback system that minimizes errors in control, guidance, and trajectory tracking. To design a fully functional GNC autopilot, a very thorough tuning of guidance parameters, controller gains, and path planning constraints needs to be carried out before implementing the system onboard an aircraft. The nonlinear dynamic of aircraft and inter-algorithmic couplings between guidance, navigation and control blocks makes the task of tuning and designing a robust autopilot challenging. The field of machine learning has a wide set of adaptable tools that are capable of learning these types of complex mappings of inputs to outputs. Artificial neural network is a widely used adaptive tool for designing control systems, but it has been

elusive in fixed-winged autopilots due to added complexity and stability issues.

In this paper we propose training a neural network autopilot with one hidden layer to unify and replace the current GNC system by directly mapping aircraft input variables (dynamic states and distances to waypoints), to control surface values. Hence the neural network learns to preserve the meaningful correlations among the individual GNC blocks by simultaneously updating its parameters (weights and biases) to output optimal control values.

The neural network autopilot is trained on supervised data from an “expert” GNC policy implemented via closed-loop 6-DOF flight simulations. The data is generated in a collaborative manner, using the neural network in training and the GNC system, by applying a modified data aggregation set, Dagger [1] algorithm. When using the standard implementation of Dagger, we find many shortcomings that prevent robust learning leading to a neural network autopilot that cannot successfully fly the aircraft. Several modifications to the algorithm are proposed to autonomously fly a fixed-wing unmanned aerial system (UAS) in different flight conditions including path following and level-turns. For each modification, the results are quantified using tracking performance, control surface magnitudes and their rates, UAS states transient behavior, and flight time.

In standard imitation learning paradigms applied to continuous control, researchers use expert policy decisions to train a controller. The control decisions of the expert are used to provide feedback to the learner such that the learner converges to the expert policy over the course of training. This type of approach has resulted in state-of-the-art controllers in a variety of different applications including in robotics and vehicle control e.g. [2]–[4]. Here we aim to adapt this approach to fixed-wing UAS where policy decisions jointly capture guidance, navigation, and control functionality. While previous research has considered using imitation learning [5] or reinforcement learning e.g. [6]–[9] for parts of UAS flight control, this work is the first attempt that the authors know of to jointly apply this type of an approach to a full GNC unification for a fixed-wing UAS.

The main contributions of this research are as follows:

- 1) A supervised learning algorithm to train artificial neural networks to behave as fully autonomous trajectory tracking autopilot for a small fixed-wing UAS. The algorithm provides a training paradigm that allows the neural network to learn a direct mapping of sensor states to control values.
- 2) Strong empirical evidence of drawbacks in standard

Daksh Shukla and Shawn Keshmiri are with the Department of Aerospace Engineering, The University of Kansas, Lawrence, KS, 66049 USA, e-mail: daksh@ku.edu and keshmiri@ku.edu.

Nicole Beckage is a Research Scientist with Intel labs, Hillsboro, OR, USA, email:nicole.beckage@intel.com.

supervised learning approaches and proposed solutions to deal with the features of complex 6-DOF dynamics with varying flight conditions. This leads to certain modifications to data collection and training, DAGger variants: Monte-Carlo-DAGger (McDagger) and Moving-Window-DAGger (MwDagger).

There has been considerable research in utilizing neural networks and applying machine learning (ML) techniques for assisting in the development of autopilots and control systems. However, our work fundamentally differs from most of the existing research as it encompasses the design of the whole autopilot software (complete GNC algorithms) through the use of a self contained neural network. The work on using ANNs for an aircraft autopilot goes back to the year 1991 [10]. Most of the research conducted in this area, has been concentrated around developing or training neural networks to act as low-level controller (a.k.a. inner-loop) [11]–[15] that compute control surface deflections when provided with guidance commands. The major shortcoming of these types of systems is that the engineer still needs to design the navigation filters, path planning, and guidance algorithms that work as outer loops for the autopilot software. Another major application of ML techniques that use neural networks has been in developing system models, also known as aircraft system identification [16]–[19]. Overall, the specific areas in which ML based ANN approximations are applied can be broadly categorized into the following areas:

- Control systems: gain scheduling [20], adaptive [21], and nonlinear control [22]. Wing rock [23], decoupled controllers [10], and optimal control for spacecraft landing [24], [25].
- Aircraft system identification or modeling of flight dynamics [26], modeling of nonlinear aerodynamics [27], and inverse plant model identification for feedback linearization [28], [29].
- Path planning and trajectory optimization [9], [30], [31].
- On-board system failure detection [32] and sensor validation [33].

Recently, there has been a shift in research towards using reinforcement learning (RL) techniques where the agent making the decision in these RL environments are neural networks. Most of the machine learning methods (especially RL) that impart some level of autonomy to drones, are mostly concentrated around the application to rotary wing aircraft e.g. [7], [8], [17], [34]–[41]. One advantage of these types of aircraft (e.g. quadcopters) can often take a “stop-think-act” approach as part of control decisions resulting in very quick alignment of neural networks to a steady “hover” state. Quadcopters can be modeled as point mass objects and do not have the same complexity of a fixed-wing aircraft, which has to maintain a minimum velocity (above stall), controlling aircraft’s angle of attack avoiding stall, and constraining maneuvers to satisfy structural load limits. This presents a stronger challenge for initial learning as well as more complicated flight dynamics in the presence of even minimal adverse environmental conditions.

For our research, a nonlinear physics-based model of a Skyhunter aircraft is used as the training environment, which is developed using Advanced Aircraft Analysis (AAA) [42] software, and further tuned using flight test data. The wing span of the aircraft is 5.9 ft, the aircraft weighs about 8.4 pounds, and flies at a cruise speed of 50.63 ft/s. It has been suitably modified to reinforce its primary structural parts, such as wing, fuselage, etc. with the addition of winglets for increasing aerodynamic performance. The details of this platform with all the modifications and an in-depth analysis of flight dynamics can be found in [43].

II. GNC TO ARTIFICIAL NEURAL NETWORKS

Artificial Neural Networks or specifically feed-forward neural networks are multi-layer perceptrons (MLPs) that in essence approximate complex relationships between inputs and outputs (mapping) and can operate as highly nonlinear functions or functions of functions (multiple layers) [44] that are parameterized by entities known as weights (w) and biases (b). Comparing the properties of neural networks with a standard GNC system, we empirically test if the whole complex process of navigation → guidance → control can be learned with high enough fidelity for a stable flight by a neural network. A theoretical GNC block diagram to a neural network transformation is depicted in Figure 1.

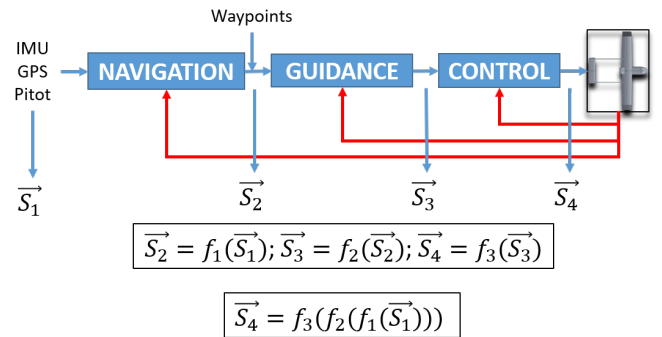


Fig. 1: GNC to an Artificial Neural Network

Consider that the navigation block has an input vector consisting of state $\vec{S}_1$, thereby outputting a vector of state $\vec{S}_2$. The state $\vec{S}_2$ can be realized to include the feedback term from the sensor measured states before becoming an input to a function f_2 and so forth. The main control output, represented here by $\vec{S}_4$, can be considered an output of a functional (function of functions) shown as $\vec{S}_4 = f_3(f_2(f_1(\vec{S}_1)))$. This is a general notion to show how a holistic GNC system, or its individual algorithms can be represented through a neural network that can learn its internal representations through data alone, without explicitly programming the logic and tuning the GNC parameters manually. Note, that the number of neural network layers required to perform this representation can vary significantly depending on the complexity of the problem at hand, the input states chosen, etc.

The “expert” GNC policy used in this research, to provide data for training the ANN autopilot is detailed in reference [45]. This GNC policy is implemented in MATLAB

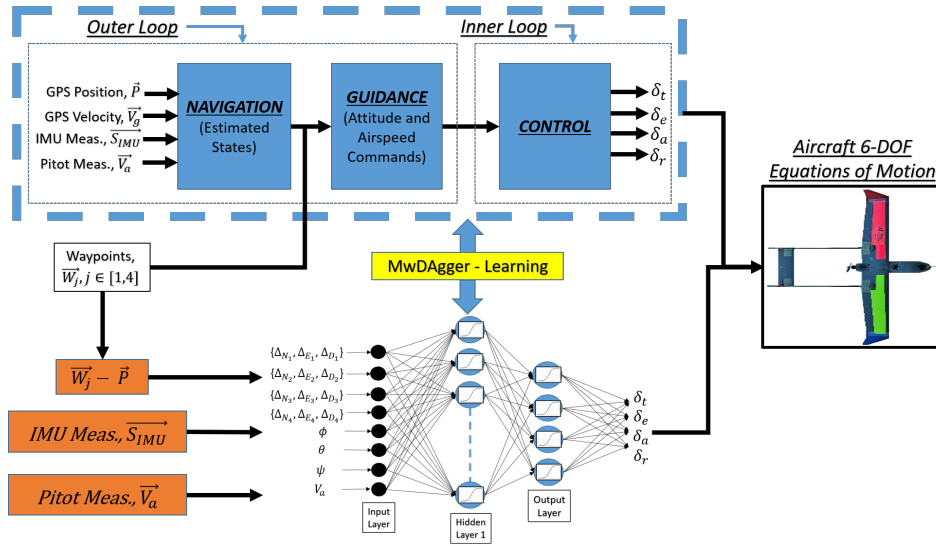


Fig. 2: The GNC: a standard autopilot for fixed wing aircraft has three components that operate in sequence, requiring different inputs (arrows in) and providing different outputs (arrows out). We introduce a neural network that receives only upstream information, bypassing many of the necessary GNC intermediate states, and directly outputs control decisions which are fed into the aircraft (6-DOF).

SIMULINK [46] simulator, and is setup to accept 3D waypoints as inputs (user defined), and follow them by exciting 6-DOF equations of motion in a closed-loop time-series flight simulation. The GNC policy tracks the waypoints based on their indices, while maintaining altitude and airspeed, and performs right or left steady-state turns based on the desired heading computed from waypoint path lines. The ANN autopilot is trained from this type of flight data, specifically a case of four waypoints laid out as vertices of an approximate rectangular box that exhibits a racetrack flight pattern and hence represents data for multiple flight control conditions: steady-state straight line, level turn, and altitude and airspeed holds.

III. ANN AUTOPILOT

The feed-forward artificial neural network used in this research consists of one hidden layer (35 neurons) and one output layer (4 neurons). The ANN autopilot directly maps higher level features (namely *sensor states* and *distance to waypoints*) to end-effector control values (see Figure 2 where the box at the bottom represents the neural network). A feed-forward neural network with one hidden layer is analytically proven to be sufficient to universally approximate any arbitrary measurable function [47], [48]. For this research, with the primary goal to investigate the unification of GNC algorithms through an ANN autopilot, the main focus is on higher level algorithmic development. Hence, the architecture shown in Figure 3 would suffice to evaluate the methodology that fulfills this goal of unification. For training purposes, MATLAB Neural Network Toolbox [49] is used. All the nodes or neurons are assigned the hyperbolic tangent, tanh, activation function. From this Neural Network Toolbox, a standard scaled conjugate gradient descent algorithm [50] is used for training the parameters of the neural network.

Designing a neural network architecture also involves choosing appropriate input variables that are uncorrelated,

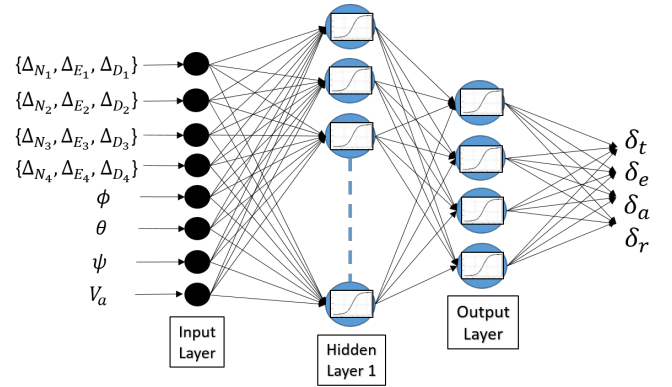


Fig. 3: ANN Autopilot Architecture

to assure it can learn a meaningful mapping to outputs. In the case of an aircraft, the inputs to the neural network are the dynamic states and outputs are generally four control variables. An aircraft involves multiple dynamic states that are pertinent to its stable operation, that should be evaluated properly before choosing them as inputs to a neural network, for example the states can be chosen from a set of velocity (V_a), angle of attack (α), side-slip angle (β), attitude angles (ϕ, θ) and attitude rates (P, Q, R). It is stated here with great emphasis, that selection of input states for this research was a great deal of challenge. In general, neural networks can learn relatively more efficiently if the cross-correlations among the inputs are as low as possible. The inputs selected to design a neural network autopilot should represent maximum causal variability across the outputs, and hence possess all the characteristics that define the physical dynamic system, in order for the model to learn quickly and, ideally, learn a set of parameters that generalize well to real world flight.

The “expert” GNC policy, details in reference [45], consists of an intelligent path planning algorithm that uses information from four waypoints (user defined) and the initial position of the aircraft (where autopilot is turned

on) to decide which waypoint to initially follow based on minimum change in heading angle. Path planning accounts for waypoint completion and switching to the next waypoint as the aircraft progresses in its flight trajectory. For our ANN autopilot, we provide the North-East-Down (NED) distances to each of the four waypoints, as inputs, so that it will learn to path plan and automatically switch around the waypoints. This ΔNED input to the ANN autopilot, which consists of 12 variables can be defined as follows:

$$\Delta K_j = \vec{W}_j - \vec{P}, \quad K \in \{N, E, D\}, \quad j \in [1, 4] \quad (1a)$$

$$\vec{W}_j = \{N_j, E_j, D_j\} \quad (1b)$$

$$\vec{P} = \{N_a, E_a, D_a\} \quad (1c)$$

Here, $\vec{W}_j$ is one of the waypoints: $\{N_j, E_j, D_j\}$ and $\vec{P}$ is the aircraft location: $\{N_a, E_a, D_a\}$. In addition to (ΔNED), it would be sufficient to provide the aircraft heading angle as part of the input representation to capture all the information needed by the path planning block in the “expert” GNC policy. The guidance algorithm of the “expert” GNC policy provides the roll angle, pitch angle and airspeed commands ($\phi_{cmd}, \theta_{cmd}, V_{a_{cmd}}$) as controller inputs. Therefore, it should be adequate to provide these aircraft states: roll, pitch and airspeed (ϕ, θ, V_a), as additional inputs to the ANN autopilot.

Using the above inputs, the neural network is trained to output control values for throttle (δ_t), elevator (δ_e), aileron (δ_a), and rudder (δ_r), with the goal to follow a race-track type trajectory composed of four waypoints shown as black dots in Figure 4.

A single training data point $x^{(i)}$ is defined as the following input vector:

$$\mathbf{x}^{(i)} = [\Delta_{N_j}^{(i)}, \Delta_{E_j}^{(i)}, \Delta_{D_j}^{(i)}, \phi^{(i)}, \theta^{(i)}, \psi^{(i)}, V_a^{(i)}], \quad j \in [1, 4] \quad (2)$$

Here, $\Delta_{N_j}, \Delta_{E_j}, \Delta_{D_j}$ are the NED distances of the aircraft from the j th desired waypoint, ϕ, θ, ψ are the attitude angles, and V_a is the airspeed. $y^{(i)}$ is output or target data value at time i and consists of the control surface variables of the aircraft, namely: throttle, elevator, aileron and rudder.

$$\mathbf{y}^{(i)} = [\delta_t^{(i)}, \delta_e^{(i)}, \delta_a^{(i)}, \delta_r^{(i)}]. \quad (3)$$

In equation 2, it is important to note that the value of j changes from 1 to 4. Therefore, the inputs comprise of 4 distances to waypoints as 3D NED vectors. With the other state inputs, the inputs add up-to a total of 16. These inputs are chosen for a specific case of four waypoints only, however, it is noted here that the value of j can be arbitrary, that is, it can go up-to a positive integer n . Hence, by changing the input architecture a neural network autopilot can be trained to fly the aircraft to follow any number of waypoints.

The input-output labelled data is generated using the “expert” GNC policy. There are multiple ways to gather this data, the simplest way would be to fly the GNC on its normal race-track flight path, wait for the flight to finish and

use the whole flight data to train the ANN autopilot. This approach detailed in section V-A is very naive, and cannot train a working ANN autopilot, due to error accumulation. Therefore, it becomes essential to use algorithms such as DAgger, that allow for data aggregation for those flight points where error accumulation starts building up. However, it is shown in the following sections that this technique also fails for complex tasks of flight paths consisting of multiple conditions (steady-state straight line, level turn, and altitude and airspeed holds). Various and incremental modifications are experimentally applied and tested to the DAgger algorithm, which finally manifests into Moving Window DAgger (MwDagger) approach that is able to train an ANN autopilot for stable and sustained flights with steady-state straight line, level turns, and airspeed and altitude holds.

IV. DAGGER FOR FIXED-WING AIRCRAFT

The basic idea behind the DAgger algorithm [1], or data aggregation, is to add new and perturbed training data, which become labeled inputs and outputs, from an expert policy. The perturbed data is novel to the neural network as well as the expert policy. In this research, the ANN is trained iteratively using the newly added data, until it is capable of flying in the flight simulation. The new data is generated in a coordinated way by the neural network and the “expert” GNC policy. First step is to perform supervised training on the ANN using fixed trajectory data generated by the GNC independently. However, this trained ANN does not perform identical to the GNC, and hence generates control outputs with small errors that accumulate over the course of a flight simulation. These slightly different outputs generated by the trained ANN, become the inputs to the aircraft 6-DOF model, which is propagated to generate a new perturbed state. These new states provide novel input to the GNC and when paired with the GNC control decisions become new labeled data from which the neural network can use for further training.

Algorithm 1 DAgger for Aircraft

```

1: Initialize state  $x^{(0)}$ ,  $D := [ ]$ ,  $T$ 
2: for  $t = 0, T$  do
3:    $y^{(t)} = \text{GNC}(x^{(t)})$ 
4:    $x^{(t+1)} = \text{6-DOF}(y^{(t)})$ 
5:    $D := D \cup \{x^{(t)}, y^{(t)}\}$ 
6: end for
7: Initialize ANN
8: while stopping criteria not met do
9:   ANN = train_model( $X, Y \subset D$ )
10:  Initialize state  $x^{(0)}$ ,  $t = 0$ 
11:  while safe( $\hat{y}^{(t)}$ ) & safe( $\hat{x}^{(t)}$ ) &  $t < T$  do
12:     $\hat{y}^{(t)} = \text{ANN}(x^{(t)})$ 
13:     $\hat{x}^{(t+1)} = \text{6-DOF}(\hat{y}^{(t)})$ 
14:     $D^* := \{\hat{x}^{(t+1)}, \text{GNC}(\hat{x}^{(t+1)})\}$ 
15:     $t = t + 1$ 
16:  end while
17:   $D = D \cup D^*$ 
18: end while

```

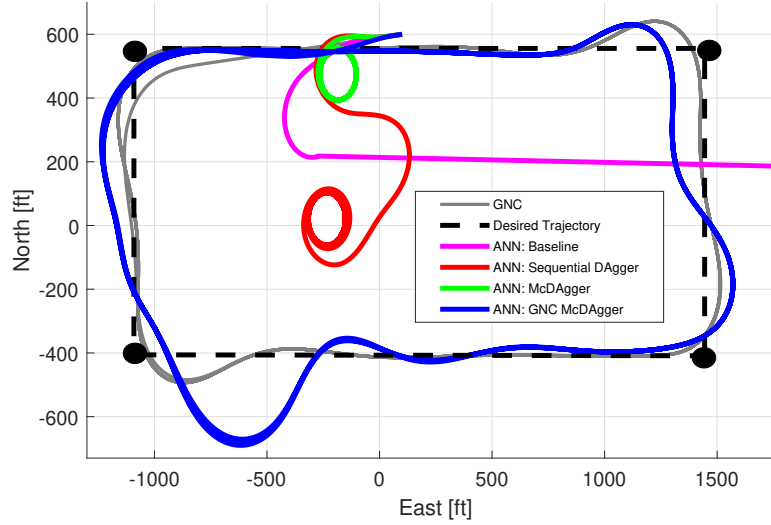


Fig. 4: All models are evaluated in a simulated flight test setting to follow the desired waypoint path (dashed line). The flight trajectories resulting from various ANN autopilot flights (solid lines: magenta, red, green, blue, and cyan) are compared to that of “expert” GNC policy trajectory (solid line: gray).

In other words, the ANN predicted controls $\hat{y}^{(t)}$ are fed to the 6-DOF aircraft model, which in turn puts the aircraft in a new state $\hat{x}^{(t+1)}$. This new data point is then fed into the GNC system such that a new training data pair, $D^* := \{\hat{x}^{(t+1)}, \mathbf{GNC}(\hat{x}^{(t+1)})\}$, is generated, which has been influenced by ANN control decisions. This new data point is then added to the original data set, and the neural network parameters are updated based on the complete data set. This process is continued iteratively until the neural network is considered trained via a stopping criterion which is defined as a small threshold difference between the GNC and ANN predicted control decisions. In addition to these more traditional stopping criteria for training, the ANN is also tested on its ability to fly the aircraft (see Figure 7 and Table I).

The detailed steps of this methodology are given in algorithm 1. The data is sampled and collected at 20Hz update rate. The first six lines of the algorithm show data collection process from the “expert” GNC policy for doing standard supervised learning. Step eight starts the data aggregation process and simultaneous training which is carried out until 1) the error between y and $\hat{y}$ is near 0, and 2) the ANN autopilot has flown the aircraft for full flight time that is “T” seconds. Steps 12, 13 and 14 show the generation of novel data which is unlikely to have been produced by the “expert” GNC policy alone. This process is repeated until any control predictions or aircraft states are deemed unsafe by the validation system. The weights of the neural network are updated in line 9 and will include control decisions from the “expert” GNC policy as new training data. The DAgger algorithm becomes the basis for the McDAgger and MwDAgger algorithms as described sections V-C and V-E, respectively.

V. SIMULATION SETUP AND DATA AGGREGATION MODELS

A general simulation flight test setup runs the “expert” GNC policy to fly the aircraft, exciting and propagating 6-DOF aircraft equations of motion (EOM). Four waypoints are setup by the user and the GNC system guides the aircraft in a racetrack pattern around these four waypoints while maintaining altitude and airspeed. The coordinates of the four waypoints are selected and converted from geodetic coordinates from a flight test location in Lawrence, Kansas, USA that is utilized for research flight tests by KU Flight Research Laboratory’s (KUFRL) team, shown as black dots in Figure 4. Evaluating the distance that the UAS has to cover in about one racetrack loop around the waypoints (≈ 7000 ft), the time taken is approximately 140 to 150 seconds at a cruise airspeed of 50.63 ft/s. Depending on the initial aircraft position with respect to the waypoints, at which the flight starts in the autopilot mode, it takes about 50 to 100 seconds for the aircraft to converge on to the desired trajectory and altitude. For collecting an initial dataset and deciding on a reasonable flight time to evaluate the ANN, about four and a half loops of racetrack trajectory was determined to be appropriate. Therefore, a flight time of 750 seconds was chosen as the target flight time for validation and data collection for the neural network autopilot.

Five different types of data aggregation algorithms are considered that train the ANN autopilot. All the ANN autopilot models produced by their respective data aggregation algorithms, are evaluated and addressed in the context of their results, as shortcomings of the preceding models motivate the changes made in later models. These models are defined as follows:

- **Baseline:** Supervised Learning with fixed data set. Only first nine lines from algorithm 1 are utilized for this model, skipping the while loops and lines 10 to 17.

- **Sequential DAGger**: See details in algorithm 1.
- **Monte Carlo DAGger (McDAGger)**: Select a random time t between 0 and T . Use the “expert” GNC policy until t instead of executing lines 12 to 14 in algorithm 1, and then start normal DAGger after time t , such that the ANN is encouraged to fly at different time points throughout the course of flight.
- **GNC McDAGger**: Select a random time t between t_{delay} and T , that is $t_{delay} \leq t < T$. The GNC is allowed to fly the aircraft to a steady-state until delay time (t_{delay}), and then the ANN autopilot is invoked at some random time following Monte-Carlo data aggregation.
- **Moving Window DAGger (MwDAGger)**: DAGger for a fixed time window of flight, allowing to focus learning on a fixed flight path (time-based). After learning a fixed path the time window is incremented and the ANN autopilot’s parameters are re-initialized, until the whole flight path is covered.

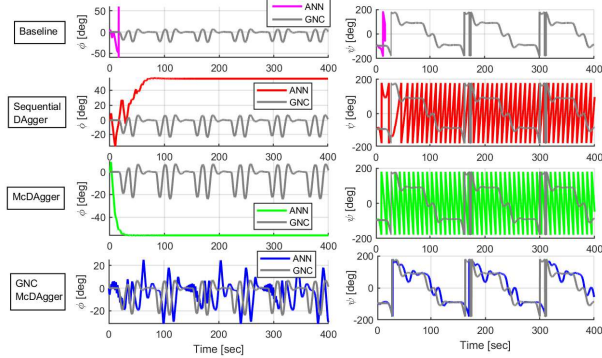


Fig. 5: Aircraft Lateral states comparison for first 4 models. Left column shows roll angle (ϕ) and right column shows yaw angle (ψ).

A. Baseline Model

In this model, the ANN is trained via supervised learning using labeled data generated independently by the “expert” GNC policy. The training is carried out until the mean squared error (MSE) is (≈ 0) or the gradient (used to update the weights during training) is below ($\approx 1e-20$). Even when applying cross-validation by dividing data into training (70%), validation (20%) and testing (10%) sets, and means to reduce over-fitting, it is found that the MSE is not a good indicator for the neural network’s ability to mimic the GNC system for a stable flight. The neural network *baseline model*, when tested on flying the aircraft via interaction with the 6-DOF EOM in closed-loop simulations, chose a flight trajectory that was nowhere close to the GNC path (see the magenta plot in Figure 4). Since the ANN autopilot, trained using the baseline algorithm, cannot predict controls with absolute zero error with respect to the “expert” GNC policy, the aircraft enters a different state in a consecutive time-step, thereby error is accumulated over closed-loop time-series simulation leading to unstable control values predicted by the baseline ANN autopilot, and the aircraft becomes unstable within 16.375 seconds. This instability in aircraft flight, using

the baseline model, is depicted in the top pane of Figures 5 and 6.

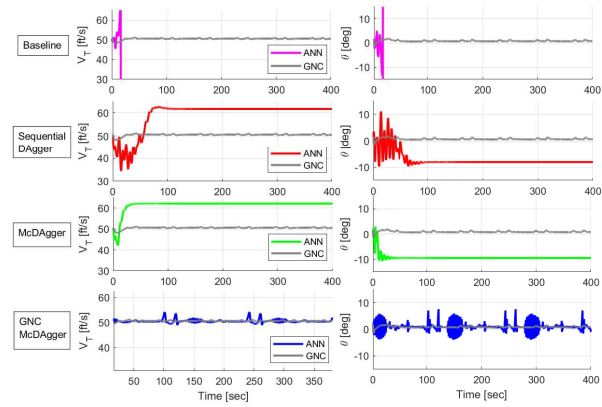


Fig. 6: Aircraft Longitudinal states comparison for all models. Left column shows total velocity (V_T) and right column shows pitch angle (θ).

B. Sequential DAGger Model

The baseline model, a standard ANN autopilot trained in a supervised manner, is not capable of mimicking the GNC controls with high enough accuracy and precision to fly the aircraft. One way to address such an issue is to allow the ANN autopilot to make control decisions that result in novel states but to return to the “expert” GNC policy in order to find out what actions are optimal in such states. *Sequential DAGger*, as explained previously, allows the neural network to fly the aircraft until it is uncontrollable or if the control inputs to 6-DOF, as outputted by the ANN, become physically unrealistic. The stopping criterion for training is set as root mean squared errors (RMSE) thresholds between the GNC controls and ANN controls: $e_t \leq 1e-3$. This number is found by manually looking at the ANN predicted controls from the baseline model and observing when the data fit is reasonably accurate without any extreme values or oscillatory behavior.

While adding data in each *sequential DAGger* iteration, it was found that the aircraft is able to fly and the ANN autopilot shows signs of learning. However after certain number of iterations, the ANN autopilot is not able to control the aircraft beyond a specific flight time (≈ 60 -70 seconds). A general trend of decrease in flight path learning is observed. This trend is shown in Figure 7, where the sequential DAGger graph is in red color. The ANN autopilot is able to fly the aircraft for complete 750 seconds in a few select cases, but it flies the aircraft on a path which is completely off-track and loses altitude continuously. The RMSE values never decrease below 0.0051 and thus the training procedure is manually stopped at DAGger iteration 239.

C. Monte Carlo DAGger (McDAGger) Model

The intermediate training and flights for the Sequential DAGger ANN autopilot have large trajectory tracking errors (see Figure 4) and high levels of instability in flight as depicted by second rows of Figures 5 and 6. Figure 8 visualizes

TABLE I: Comparison of ANN GNC autopilots trained in different imitation learning paradigms. Model evaluation focuses on training feasibility and flight time during validation and testing.

Model	Dagger Iterations (Training)	Training Examples	Training Time	Slope of Flight Times	Flight Time (seconds)
Baseline	N.A.	1.50e4	0h 4m	N.A.	16.375
Seq. DAgger	239	6.08e5	7h 35m	-0.54132	750
McDagger	279	1.27e5	1h 52m	0.12801	48.5
GNC McDagger	325	> 1e6	9h 14m	0.57104	800+

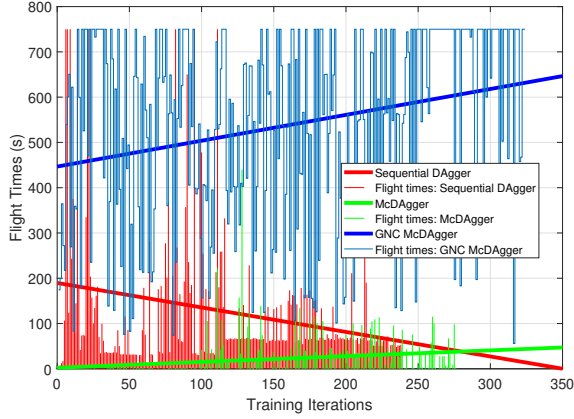


Fig. 7: Flight time slopes for DAgger training, comparison for all models.

the data aggregation steps (lines 12 to 14 in algorithm 1) such that the light to darker color transitions indicate the increment in successive DAgger training iterations. As can be seen from the plot, some of the light colored trajectories show significant amount of flight time (500 - 750 seconds), however, the darker colored trajectories are limited to around (60 - 70 seconds). One of the major reasons that the training appears to converge around 60 seconds, and the slope of flight time is negative, is apparent by looking at Figure 8, that is, the additional data being aggregated to the original data set is non-uniformly distributed around the four waypoint trajectory. Or in other words, the data being added is more or less concentrated around the first 60 seconds of the flight path resulting in the model over-fitting to the earliest part of the flight. To overcome this issue, a straight-forward modification to the original DAgger algorithm was applied, in which the aircraft starts data addition at a different state at each iteration, selected by randomly sampling the starting time of the simulation, this would result in updating lines 10 to 16 in algorithm 1.

The modification of choosing a random time to initialize data collection, and thus a corresponding random state ($x^{(t)}$) of the aircraft that does not have the accumulated errors from all previous flight decisions, leads to the terminology: Monte Carlo DAgger. Surprisingly, the flight times per McDagger iteration were found to often be even shorter than for the Sequential DAgger autopilot but it is noted that the slope (e.g. the trend in flight length over training time) is at least positive (see the green line in Figure 7), indicating that with longer training or more computational power this model may have eventually converged to the required 750 seconds of flight.

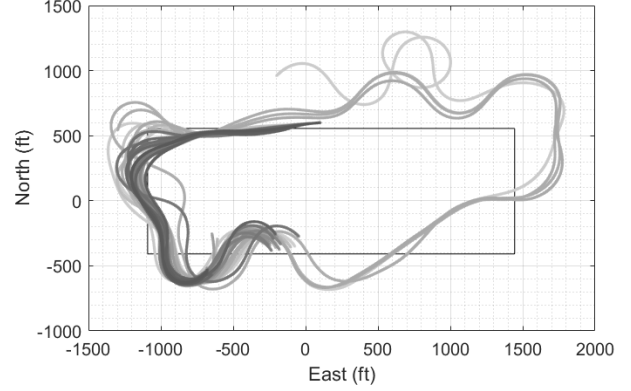


Fig. 8: Sequential DAgger: Intermediate trajectories flown by ANN during data aggregation.

D. GNC McDagger Model

The “expert” GNC policy used to generate correct input-output mapping data, is designed in such a way that it does not react to errors abruptly and always ensures a very smooth transition if initialized from a large state or control error. When the simulation for GNC and 6-DOF, is initialized at a random state for the aircraft, it will always have a period of transition (t_{delay}) to converge to the correct cruise altitude and hence the desired trajectory. In other words, even if the simulator is initialized at an exact point on a previously flown trajectory, the GNC outputted control values will not be the same as the controls on the original path, and hence the aircraft flying with the GNC policy does not follow a particular pattern of flying when initialized randomly and hence it is not on-policy. This causes data distribution mismatch issues, because the neural network is learning from policy decisions that are variant even for the same set of states. Hence the flight time for McDagger autopilot stays around 40 seconds (Figure 7).

If data addition is desired to be initialized at a random location on the original trajectory, the GNC induced flight behavior should be on policy, based on consistent control decisions relevant to the flight trajectory the ANN is trying to learn (i.e. post initialization phase of the GNC). This is addressed by initializing the simulator from a fixed initial condition ($x^{(0)}$: aircraft state) but with a variable start to when data aggregation is engaged. The GNC flies from an initial condition until a randomized time t which is at least delayed for the period of transition (t_{delay}). After the GNC has flown the aircraft up to this random point, then the ANN is invoked to take over the flight and McDagger data addition resumes. This modification to McDagger algorithm

ensures proper data aggregation and is mainly applied to work around the specific limitations of our “expert” GNC policy. This modified version of McDagger algorithm is termed as model: GNC MCDagger and requires two main changes to algorithm 1, namely t is selected at random after adding delay time and the GNC is initialized to run from $x^{(0)}$ until $t - 1$. The slope of flight times during GNC McDagger iterations is shown in Figure 7 in blue color and indicates that this training procedure is allowing the ANN autopilot to increase flight time steadily by adding data that is useful for ANN control learning. The flight times are consistently increasing and reaching up to 750 seconds which is the maximum flight path time. Additionally, while the model is not following the GNC trajectory exactly, the ANN learning curves, depicted in Figure 9 do suggest that the flights are getting longer and more closely aligned with the desired trajectory as training proceeds.

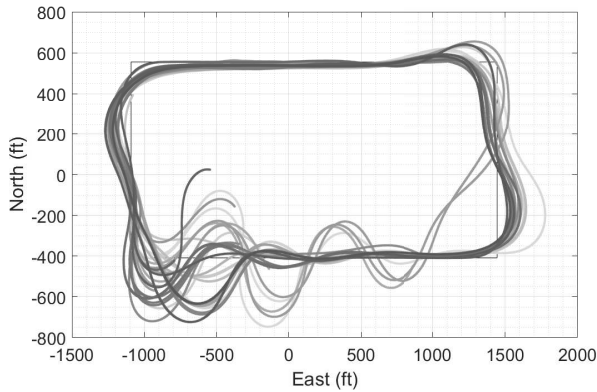


Fig. 9: GNC McDagger: Intermediate trajectories flown by ANN during training: ANN Learning Curves

On comparing the learning curves from models: Sequential Dagger and GNC McDagger (Figures 8, 9), it can be clearly observed that the flight paths during learning are closer to the desired trajectory for the GNC McDagger model. After training is completed, each ANN autopilot is subjected to a simulated flight test, starting the aircraft at the same initial state used for training and without any involvement of GNC. The aircraft 6-DOF equations are excited by ANN outputted control values in a time-series closed-loop simulation. We can then compare the trajectory as flown by all the ANN autopilot models (trained via DAgger variants) with the “expert” GNC trajectory. Results for all the five models are shown in Figure 4. It is observed that the overall simulated flight path for only the GNC McDagger ANN autopilot is satisfactory with respect to the GNC simulated flight path. This ANN autopilot has approximately ± 20 ft of North distance errors along the East-West waypoint legs of the desired path, and approximately ± 80 ft of East distance errors along the North-South legs. The largest tracking errors are observed around the South-West corner of the desired trajectory, which is about 267 ft. These trajectory errors are not ideal in terms of a realistic flight test and must be improved before putting this ANN autopilot on an actual aircraft, but a measurable progress is shown in improving

imitation learning in order to construct a ANN autopilot.

For the simulated flight tests with the Baseline, Sequential DAgger, McDagger and GNC McDagger neural networks, the aircraft states are analyzed in both longitudinal and lateral directions which are depicted in Figures 6 and 5, respectively. Figure 6 shows longitudinal states, namely: total velocity and pitch angle of the aircraft for all four ANN autopilot models. Oscillations are observed in both the aircraft states for the GNC McDagger neural network. There are low frequency (≈ 1.2 Hz) pitch oscillations observed on the aircraft. It is also worth mentioning that the aircraft is able to hold the desired cruise altitude of 300 ft.

In the bottom panel of Figure 5, some lateral states, namely, roll angle and yaw (or $\approx$ heading) angle, are shown for the GNC McDagger ANN autopilot and compared to the GNC simulation states. There are again low frequency oscillations observed in the roll angle of the aircraft, which are not ideal for a realistic flight, and must be addressed before putting this autopilot on a real aircraft.

Table I shows the statistics of training for the first four models. In terms of computational cost, the GNC McDagger is the most expensive, and it also uses the largest amount of data which requires interactions directly with the GNC, to train the ANN autopilot.

E. Moving Window DAgger: Addressing Oscillations

The GNC McDagger ANN autopilot model was successfully able to fly the aircraft in simulation, but with observed low frequency state oscillations. This indicates that the ANN autopilot was unable to make correct control decisions for small state perturbations, and would react only when the aircraft state (and hence 2D trajectory) would drift enough for the ANN to take corrective action. To mitigate this issue, the sequential data aggregation was applied to a fixed time-window, and the ANN was trained iteratively over this window until a stopping criterion was met. This aggregated data along a short time window is saved in a data buffer, and the ANN parameters are reinitialized. The time-window for data aggregation is increased, and new data is added to the data buffer, and the ANN is trained for this time-window. The process is repeated until the whole flight path is covered by the time-window, increasing up-to the maximum flight time. Algorithm 2 shows the details of the Moving Window DAgger (MwDagger) algorithm applied to a fixed-wing aircraft with an “expert” GNC policy. It took 753 DAgger iterations, and the algorithm aggregates a total of $1.52e5$ data points, taking a total of about 8 hours and 55 minutes. This ANN autopilot is able to fly indefinitely around the waypoint path, maintaining stable states at all times.

The longitudinal and lateral-directional (states and control) are shown in Figures 10 and 11 respectively. It can be clearly observed that there are no oscillations in the aircraft states and the control outputs from the ANN are very smooth and on par with the GNC controls. Velocity tracking is reasonable and within an error margin of 0.5 ft/s, which is very similar in behavior to the “expert” GNC policy.

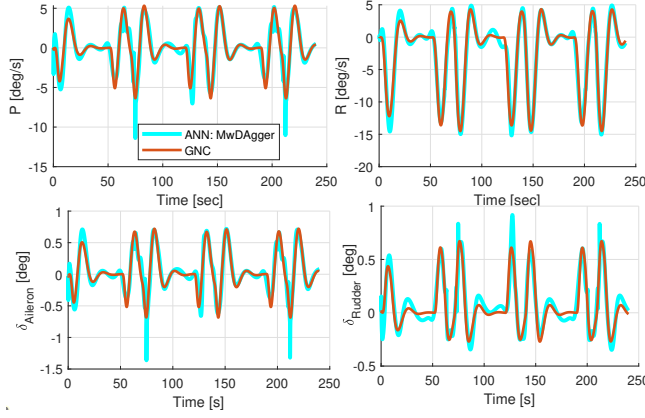


Fig. 10: M_3 : Moving Window DAgger ANN-Simulation Flight Test: Lateral States and Controls

Algorithm 2 Moving Window DAgger for Aircraft

```

1: Initialize state  $x^{(0)}$ ,  $D := [ ]$ ,  $T$ 
2: for  $t = 0, T$  do
3:    $y^{(t)} = \mathbf{GNC}(x^{(t)})$ 
4:    $x^{(t+1)} = \mathbf{6-DOF}(y^{(t)})$ 
5:    $D := D \cup \{x^{(t)}, y^{(t)}\}$ 
6:   Initialize  $w$  and  $w_s$ 
7: end for
8: while stopping criteria not met do
9:   Initialize ANN
10:   $\text{ANN} = \text{train\_model}(X, Y \subset D)$ 
11:  Initialize state  $x^{(0)}$ ,  $t = 0$  and  $t_f = w$ 
12:  while  $\text{safe}(\hat{y}^{(t)})$  &  $\text{safe}(\hat{x}^{(t)})$  &  $t < t_f$  do
13:     $\hat{y}^{(t)} = \text{predict\_ANN}(x^{(t)})$ 
14:     $\hat{x}^{(t+1)} = \mathbf{6-DOF}(\hat{y}^{(t)})$ 
15:     $D^* := \{\hat{x}^{(t+1)}, \mathbf{GNC}(\hat{x}^{(t+1)})\}$ 
16:     $t = t + 1$ 
17:  end while
18:  if  $\text{RMSE}\{\hat{y}^{(t)}, \mathbf{GNC}(\hat{x}^{(t+1)})\} \leq \Delta_c$  then
19:     $w = w + w_s$ 
20:  end if
21:   $D = D \cup D^*$ 
22: end while

```

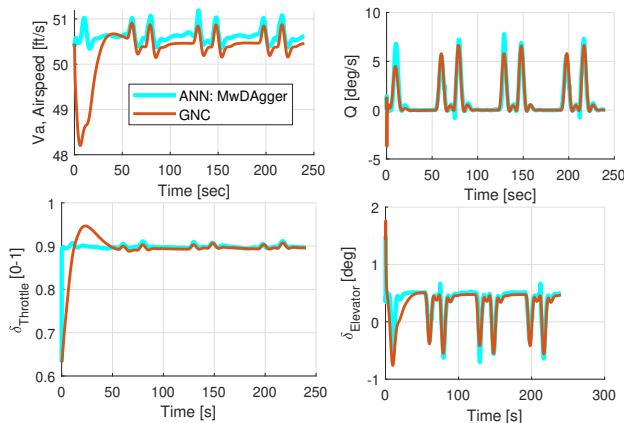


Fig. 11: M_3 : Moving Window DAgger ANN-Simulation Flight Test: Longitudinal States and Controls

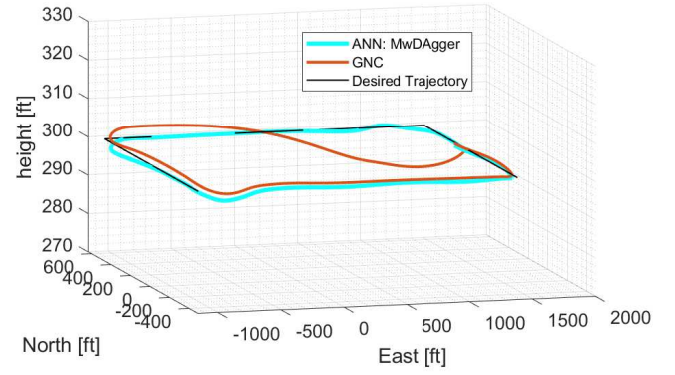


Fig. 12: M_3 : Moving Window DAgger - 3-Dimensional Trajectory Comparison with GNC

In Figure 12, the ANN autopilot's and GNC policy's flight trajectories are compared. It is very interesting to note that the GNC flight makes the aircraft lose altitude by about 15 ft in the beginning, but the ANN autopilot is able to hold the aircraft altitude throughout the path.

VI. CONCLUSION

The primary goal of this research was to explore the possibility of training an artificial neural network autopilot to mimic and unify guidance, navigation, and control algorithms of an existing autopilot that is known to succeed in a variety of real-world flight conditions. This research demonstrates that the standard supervised learning techniques are insufficient to train an end-to-end ANN autopilot that can fly a highly nonlinear 6-DOF model of a fixed-wing aircraft. While the GNC has access to the intermediate calculations that allows for parameter adjustments to prevent an unstable flight, the ANN autopilot must predict controls directly from the aircraft's raw measured states. The potential of this ANN autopilot to achieve the difficult task of aircraft flight through software in the loop simulations is shown and validated for multiple flight conditions: steady-state straight line, level-turn, and altitude and airspeed holds. A unique learning algorithm, GNC McDagger, was developed to address the challenges of data distribution mismatch across a wide flight envelope. However, the algorithm introduced low frequency oscillations, which were eliminated through a novel approach called: MwDagger learning.

ACKNOWLEDGMENT

This work was completed with funding from NASA and DARCorporation at the University of Kansas Flight Research Laboratory.

REFERENCES

- [1] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, 2011, pp. 627–635.
- [2] B. D. Argall, S. Chernova, M. Veloso, and B. Browning, "A survey of robot learning from demonstration," *Robotics and autonomous systems*, vol. 57, no. 5, pp. 469–483, 2009.

- [3] Y. Pan, C.-A. Cheng, K. Saigol, K. Lee, X. Yan, E. Theodorou, and B. Boots, "Agile off-road autonomous driving using end-to-end deep imitation learning," *arXiv preprint arXiv:1709.07174*, 2017.
- [4] O. Aşık, B. Görer, and H. L. Akin, "End-to-end deep imitation learning: Robot soccer case study," *arXiv preprint arXiv:1807.09205*, 2018.
- [5] S. Ross, N. Melik-Barkhudarov, K. S. Shankar, A. Wendel, D. Dey, J. A. Bagnell, and M. Hebert, "Learning monocular reactive uav control in cluttered natural environments," in *2013 IEEE international conference on robotics and automation*. IEEE, 2013, pp. 1765–1772.
- [6] K. Kersandt, "Deep reinforcement learning as control method for autonomous uavs," Master's thesis, Universitat Politècnica de Catalunya, 2018.
- [7] S. Choi, S. Kim, and H. J. Kim, "Inverse reinforcement learning control for trajectory tracking of a multirotor uav," *International Journal of Control, Automation and Systems*, vol. 15, no. 4, pp. 1826–1834, 2017.
- [8] J. Hwangbo, I. Sa, R. Siegwart, and M. Hutter, "Control of a quadrotor with reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 2, no. 4, pp. 2096–2103, 2017.
- [9] B. Zhang, Z. Mao, W. Liu, and J. Liu, "Geometric reinforcement learning for path planning of uavs," *Journal of Intelligent & Robotic Systems*, vol. 77, no. 2, pp. 391–409, 2015.
- [10] T. Troudet, S. Garg, and W. Merrill, "Neural network application to aircraft control system design," in *Navigation and Control Conference*, 1991, p. 2715.
- [11] C. Ha, "Neural networks approach to aiaa aircraft control design challenge," *Journal of Guidance, Control, and Dynamics*, vol. 18, no. 4, pp. 731–739, 1995.
- [12] M. R. Napolitano and M. Kincheloe, "On-line learning neural-network controllers for autopilot systems," *Journal of Guidance, Control, and Dynamics*, vol. 18, no. 5, pp. 1008–1015, 1995.
- [13] M. Ryan and M. Reid, "Learning to fly: An application of hierarchical reinforcement learning," in *In Proceedings of the 17th International Conference on Machine Learning*. Citeseer, 2000.
- [14] H. Baomar and P. J. Bentley, "An intelligent autopilot system that learns piloting skills from human pilots by imitation," in *Unmanned Aircraft Systems (ICUAS), 2016 International Conference on*. IEEE, 2016, pp. 1023–1031.
- [15] G. C. Cottrill, "Neural network autopilot system for a mathematical model of the boeing 747," WEST VIRGINIA UNIV MORGANTOWN, Tech. Rep., 1998.
- [16] J. Manerowski and D. Rykaczewski, "Modelling of uav flight dynamics using perceptron artificial neural networks," *Journal of theoretical and applied mechanics*, vol. 43, 2005.
- [17] A. Punjani and P. Abbeel, "Deep learning helicopter dynamics models," in *Robotics and Automation (ICRA), 2015 IEEE International Conference on*. IEEE, 2015, pp. 3223–3230.
- [18] A. Roudbari and F. Saghaei, "Intelligent modeling and identification of aircraft nonlinear flight," *Chinese Journal of Aeronautics*, vol. 27, no. 4, pp. 759–771, 2014.
- [19] M. K. Samal, S. Anavatti, and M. Garratt, "Neural network based system identification for autonomous flight of an eagle helicopter," *IFAC Proceedings Volumes*, vol. 41, no. 2, pp. 7421–7426, 2008.
- [20] A. A. Bhatti, "Neural network based control system design of an advanced fighter aircraft," in *Telesystems Conference, 1991. Proceedings. Vol. 1., NTC'91., National*. IEEE, 1991, pp. 1–6.
- [21] S. Balakrishnan and V. Biega, "Adaptive-critic-based neural networks for aircraft optimal control," *Journal of Guidance, Control, and Dynamics*, vol. 19, no. 4, pp. 893–898, 1996.
- [22] A. J. Calise and R. T. Rysdyk, "Nonlinear adaptive flight control using neural networks," *IEEE control systems*, vol. 18, no. 6, pp. 14–25, 1998.
- [23] S. N. Singh, W. Yirn, and W. R. Wells, "Direct adaptive and neural control of wing-rock motion of slender delta wings," *Journal of Guidance, Control, and Dynamics*, vol. 18, no. 1, pp. 25–30, 1995.
- [24] C. Sánchez-Sánchez and D. Izzo, "Real-time optimal control via deep neural networks: study on landing problems," *Journal of Guidance, Control, and Dynamics*, vol. 41, no. 5, pp. 1122–1135, 2018.
- [25] L. Cheng, Z. Wang, Y. Song, and F. Jiang, "Real-time optimal control for irregular asteroid landings using deep neural networks," *arXiv preprint arXiv:1901.02210*, 2019.
- [26] U. J. Pesonen, J. E. Steck, K. Rokhsaz, H. S. Bruner, and N. Duerksen, "Adaptive neural network inverse controller for general aviation safety," *Journal of Guidance, Control, and Dynamics*, vol. 27, no. 3, pp. 434–443, 2004.
- [27] J. Steck, K. Rokhsaz, J. Steck, and K. Rokhsaz, "Some applications of artificial neural networks in modeling of nonlinear aerodynamics and flight dynamics," in *35th Aerospace Sciences Meeting and Exhibit*, 1997, p. 338.
- [28] C. C. Jorgensen, "Direct adaptive aircraft control using dynamic cell structure neural networks," 1997.
- [29] B. S. Kim and A. J. Calise, "Nonlinear flight control using neural networks," *Journal of Guidance, Control, and Dynamics*, vol. 20, no. 1, pp. 26–33, 1997.
- [30] J. F. Horn, E. M. Schmidt, B. R. Geiger, and M. P. DeAngelo, "Neural network-based trajectory optimization for unmanned aerial vehicles," *Journal of Guidance, Control, and Dynamics*, vol. 35, no. 2, pp. 548–562, 2012.
- [31] J. Horn, B. Geiger, and E. Schmidt, "Use of neural network approximation in multiple-unmanned aerial vehicle trajectory optimization," in *AIAA Guidance, Navigation, and Control Conference*, 2009, p. 6103.
- [32] M. R. Napolitano, C. Neppach, V. Casdorph, S. Naylor, M. Innocenti, and G. Silvestri, "Neural-network-based scheme for sensor failure detection, identification, and accommodation," *Journal of Guidance, Control, and Dynamics*, vol. 18, no. 6, pp. 1280–1286, 1995.
- [33] G. Campa, M. L. Fravolini, M. Napolitano, and B. Seanor, "Neural networks-based sensor validation for the flight control system of a b777 research model," in *American Control Conference, 2002. Proceedings of the 2002*. IEEE, 2002, pp. 412–417.
- [34] D. Gandhi, L. Pinto, and A. Gupta, "Learning to fly by crashing," *arXiv preprint arXiv:1704.05588*, 2017.
- [35] O. Andersson, M. Wzorek, and P. Doherty, "Deep learning quadcopter control via risk-aware active learning," in *AAAI*, 2017, pp. 3812–3818.
- [36] D. K. Kim and T. Chen, "Deep neural network for real-time autonomous indoor navigation," *arXiv preprint arXiv:1511.04668*, 2015.
- [37] A. Coates, P. Abbeel, and A. Y. Ng, "Autonomous helicopter flight using reinforcement learning," in *Encyclopedia of Machine Learning*. Springer, 2011, pp. 53–61.
- [38] T. Dierks and S. Jagannathan, "Output feedback control of a quadrotor uav using neural networks," *IEEE transactions on neural networks*, vol. 21, no. 1, pp. 50–66, 2010.
- [39] P. Abbeel, A. Coates, and A. Y. Ng, "Autonomous helicopter aerobatics through apprenticeship learning," *The International Journal of Robotics Research*, vol. 29, no. 13, pp. 1608–1639, 2010.
- [40] P. Abbeel, A. Coates, T. Hunter, and A. Y. Ng, "Autonomous autorotation of an rc helicopter," in *Experimental Robotics*. Springer, 2009, pp. 385–394.
- [41] A. Coates, P. Abbeel, and A. Y. Ng, "Learning for control from multiple demonstrations," in *Proceedings of the 25th international conference on Machine learning*. ACM, 2008, pp. 144–151.
- [42] A. Design and R. Corporation, *Advanced Aircraft Analysis*, 3rd ed., Lawrence, KS.
- [43] A. T. Blevins, H. Benyamen, G. Godfrey, D. Shukla, and B. Kim, "Analysis and verification of cost-effective design modifications to commercially available fixed-wing unmanned aerial vehicle to improve performance, stability and control characteristics, and structural integrity," in *2018 AIAA Information Systems-AIAA Infotech@Aerospace*, 2018, p. 2261.
- [44] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.
- [45] A. R. Kim, D. Shukla, A. Blevins, S. Keshmiri, and M. Ewing, "Validation and verification of gnc for large fixed-wing unmanned aerial system in unstructured and noisy environment," in *Unmanned Aircraft Systems (ICUAS), 2018 International Conference on*. IEEE, 2018.
- [46] MATLAB, 9.7.0.1190202 (R2018a). Natick, Massachusetts: The MathWorks Inc., 2018.
- [47] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural networks*, vol. 2, no. 5, pp. 359–366, 1989.
- [48] G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of control, signals and systems*, vol. 2, no. 4, pp. 303–314, 1989.
- [49] "Matlab deep neural network toolbox v11.1," 9.4.0.813654 (R2018a), the MathWorks, Natick, MA, USA.
- [50] M. F. Møller, "A scaled conjugate gradient algorithm for fast supervised learning," *Neural networks*, vol. 6, no. 4, pp. 525–533, 1993.